Assignment #2 - Neural Networks from Scratch

In this assignment you will implement a fully connected neural network model from scratch in Python. You will then use your implementation to analyze various datasets and to explore different aspects of the deep learning process.

I. Neural Network Implementation (60 *points)*

Implement a fully connected neural network in Python from scratch. I have provided you with the basic code skeleton with the functions you will need to implement. You may not use any libraries other than those specified at the top of each file, and you may not change any of the function signatures. Make sure to use the supplied conda environment in **a2_env.yml**, otherwise you might accidentally use different libraries, and your results may differ. I have provided tests for several of the functions you are required to implement in **tests.py**. You are required to pass these tests in order to earn credit; however, they are not meant to be comprehensive (it's possible for your functions to still be incorrect even if they pass all tests). You may not modify any code in these files other than in the functions you are required to implement.

The following functions must be completed in **utils.py**. More detailed instructions for these implementations can be found in the Python file.

1. sigmoid_derivative (3 points)
2. softmax (4 *points)*
3. cross_entropy_loss (5 points)
4. one_hot_encode (3 *points)*

The following functions must be completed in **neural_network.py**. More detailed instructions for these implementations can be found in the Python file.

1. initialize_parameters (5 *points)*
2. forward_pass *(10 points)*
3. backward_pass *(10 points)*
4. train (15 *points)*
5. predict *(5 points)*

II. Analysis *(40 points)*

In this section you will use your neural network implementation to analyze datasets. All code for this section must be written in **analysis.py**. All figures and written responses should be included in the provided **analysis** file. You may not use any Python libraries that are not already included for you at the top of the file. All figures that are included in your analysis should be saved to the directory ***../plots/*** when the functions are called. All your figures should be reproducible when I run your code. If I cannot reproduce the figures exactly as they are shown in your analysis file, then you will receive no credit for these figures.

1. Linearly Separable Dataset (5 *points)*

    - Train a neural network model on the **linearly separable** dataset using the hyperparameters specified in **analysis.py**.
    - Create a figure of the decision boundary for this model overlaid on the training dataset. You may use the **plot_decision_boundary** function provided in **utils.py**.
    - Print to the console the accuracy achieved by your model on the training dataset.

2. Learning Non-Linear Relationships *(10 points)*

    - Train three neural network models on the **concentric circles** dataset to compare the effect of modifying network architecture and activation functions on nonlinearly separable datasets. The hyperparameters for each neural network model are specified in **analysis.py**.
    - For each model, create a figure of the decision boundary overlaid on the training dataset.
    - For each model, print to the console the training accuracy.
    - Provide a written analysis of your results. Comment on the decision boundaries. Comment on the training accuracies. What do these results demonstrate about the role of hidden layers and nonlinear activation functions in neural networks?

3. Model Complexity *(10 points)*

    - Train six neural network models on the **two moons** dataset to compare the effect of model complexity on model training. The hyperparameters for each neural network model are specified in **analysis.py**.
    - For each model, create a figure of the decision boundary overlaid on the training dataset.
    - For each model, print to the console the training accuracy.
    - For each model, print to the console the number of parameters in the model.
    - For each model, print to the console the training time.
    - Provide a written analysis of your results. Comment on the decision boundaries. Comment on the training accuracies. Comment on the number of parameters for each model. Comment on the training time for each model. What do these results demonstrate about the role of model complexity on neural network training?

4. Breast Cancer Classification *(15 points)*

    - Train a neural network model on the **Breast Cancer Wisconsin Dataset** using the hyperparameters specified in **analysis.py**. The dataset has been split into training, validation, and testing sets. Save the model parameters after each epoch.
    - Create a figure of the loss curves, displaying the training loss and validation loss as a function of the number of epochs.
    - Create a figure displaying the training and validation accuracy as a function of the number of epochs.
    - Determine the optimal number of epochs based on the validation loss, and set the model parameters to those from the optimal epoch.
    - Print to the console the optimal number of epochs and the training, validation, and testing accuracies of the model at the optimal epoch.
    - Print to the console the confusion matrix for the testing set, as well as the precision, recall, and F1-score, treating malignant tumors as the positive class.

- Provide a written analysis of your results. Comment on the loss and accuracy curves. Does the model exhibit signs of overfitting, underfitting, or neither? How was the optimal number of epochs determined? How well does the model generalize to unseen data? Comment on the evaluation metrics and their implications for this dataset. How could the model or evaluation process be improved?

III. What to turn in?

You must submit the following files to Canvas.

1) **analysis.py** - Running this file should save all of the figures from your analysis in the **plots** directory. If these figures do not match exactly the figures provided in your report you will receive no credit. All figures must be formatted professionally, e.g. include a relevant title, axes labels, legend, etc.
2) **utils.py**
3) **neural_network.py**
4) **analysis.pdf** - This file must be converted to a pdf. Do not include any extra slides other than the ones provided.

IV. Important Notes

- Make sure all of the figures in your analysis are reproducible exactly as they appear in your analysis slides. I will be running your code to ensure this. If your figures are not reproducible you will receive no credit for those figures.

- This is an individual assignment and must be completed independently. You may discuss the assignment with other students, but all code and written analysis must be done independently.